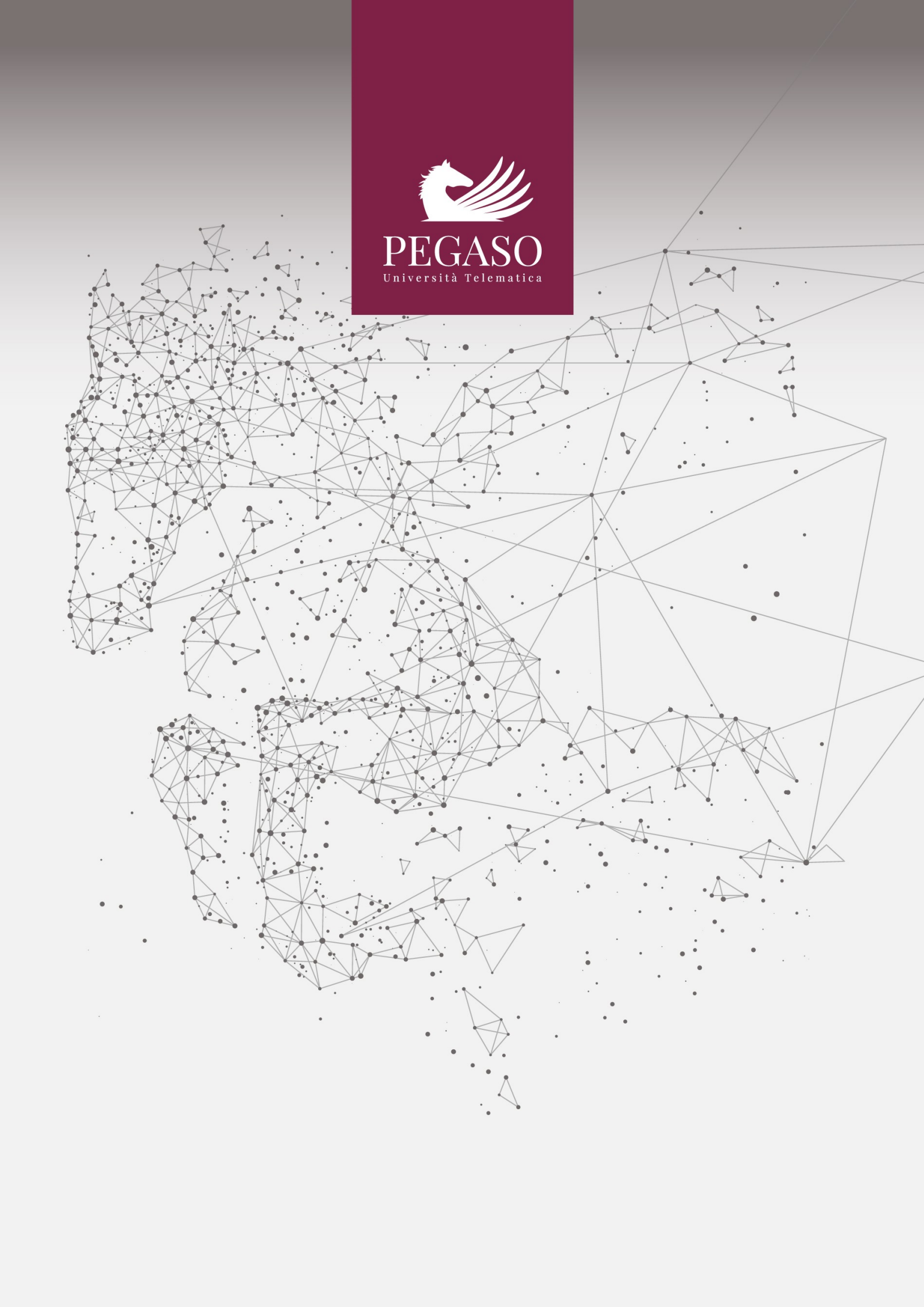

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3

2. MANIPOLAZIONE AVANZATA DI FRAME E SERIE 4

2.1 JOIN 4
2.2 DISCRETIZZAZIONE 6

3. GENERAZIONE DI FRAME E OPERAZIONI CON I FILE 9

3.1 GENERAZIONE DI DATAFRAME 9
3.2 SALVATAGGIO DI DATAFRAME SU FILE 12

4. GRAFICI CON PANDAS 14

5. ESERCIZIO DI DATA SCIENCE 17

BIBLIOGRAFIA 27

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presentano due operazioni di manipolazione avanzata di frame e serie, la congiunzione (join) e la discretizzazione. Si illustrano, inoltre, sia i metodi possibili di generazione dei DataFrame, inclusi quelli relativi alla lettura da file (file csv, file Excel, database), sia le diverse opzioni di salvataggio di DataFrame su file. Si presenta, poi, la rappresentazione grafica con Pandas completamente compatibile con la libreria Matplotlib. Infine, si riporta un esercizio di Data Science basato sull'analisi di transazione bancarie che permette di mettere in pratica la maggioranza delle operazioni analizzate.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Manipolazione avanzata di frame e serie

2.1 Join

Pandas propone nella Data Science tutte le operazioni tipiche che si svolgono sui database. Tra queste operazioni c'è anche il **JOIN** (congiunzione) di dati di **DataFrame differenti.**

Si metteranno, quindi, in rapporto tra loro i dati dei campi di più DataFrame e si unificheranno le righe per cui esiste un'associazione. Come esempio, si creano tre DataFrame con dati di fantasia:

1. ***conti*** che contiene numero di contro e intestatario di una serie di conti correnti;
2. ***estinzioni*** che raccoglie le date di estinzione dei conti correnti che sono stati chiusi (solo i rapporti interrotti saranno registrati in questo DataFrame che, quindi, non conterrà tutti i conti presenti nel primo DataFrame);
3. ***saldi*** che raccoglie i saldi di tutti i conti correnti ancora attivi.

```
import pandas as pd

conti = pd.DataFrame({'conto':['111100','111200','111300','111400','111500'],\
                      'intestatario':['Paolo Verdi','Ida Neri','Simona Bianchi',\
                                      'Ugo Rossi','Mirko Verdi']})

estinzioni = pd.DataFrame({'conto':['111100','111400'],\
                      'data':['2010-09-22','2004-11-12']})

saldi = pd.DataFrame({'conto':['111200','111300','111500'],\
                      'saldo':[120000,3500,23000]})

print("Conti")
print(conti)
print()
print("Estinzioni")
print(estinzioni)
print()
print("Saldi")
print(saldi)
print()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
Conti
    conto    intestatario
0  111100     Paolo Verdi
1  111200        Ida Neri
2  111300  Simona Bianchi
3  111400       Ugo Rossi
4  111500     Mirko Verdi

Estinzioni
    conto        data
0  111100  2010-09-22
1  111400  2004-11-12

Saldi
    conto   saldo
0  111200  120000
1  111300    3500
2  111500   23000
```

Indipendentemente dal fatto che quella appena vista rappresenti o meno una buona organizzazione dei dati, vogliamo eseguire l'operazione di JOIN per allineare le informazioni. Il metodo offerto da Pandas per eseguire dei JOIN prende il nome di **merge**() e richiede i seguenti argomenti:

- DataFrame da incrociare;
- nome del campo su cui eseguire l'incrocio;
- modalità di esecuzione del JOIN.

Realizziamo, quindi, un join tra i dati di *conti* e di *saldi* sul campo 'conto' come indicato di seguito.

```
j = pd.merge(conti, saldi, on='conto')
print(j)
```

```
    conto    intestatario   saldo
0  111200        Ida Neri  120000
1  111300  Simona Bianchi    3500
2  111500     Mirko Verdi   23000
```

Il DataFrame *conti* ha cinque righe e *saldi* ne ha tre: il risultato che ne deriva ha tre righe anch'esso in quanto conterrà solo i record in cui è stato possibile trovare un'associazione come mostrato nella Figura seguente in codice colore.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Conti

	conto	intestatario
0	111100	Paolo Verdi
1	111200	Ida Neri
2	111300	Simona Bianchi
3	111400	Ugo Rossi
4	111500	Mirko Verdi

Saldi

	conto	saldo
0	111200	120000
1	111300	3500
2	111500	23000

Per mettere in relazione *conti* e *saldi*, facendo comunque apparire tutti i conti sia attivi che estinti, si dovrà specificare di voler eseguire un JOIN esterno (in questo caso LEFT OUTER JOIN) attraverso la proprietà *how*. In questo caso, il DataFrame risultante includerà:

- tutti i record del DataFrame di sinistra;
- per le mancate corrispondenze *NaN* nei campi del DataFrame di destra.

```
j = pd.merge(conti, saldi, on='conto', how='left')
print(j)
```

```
    conto    intestatario     saldo
0  111100     Paolo Verdi       NaN
1  111200        Ida Neri  120000.0
2  111300  Simona Bianchi    3500.0
3  111400       Ugo Rossi       NaN
4  111500     Mirko Verdi   23000.0
```

2.2 Discretizzazione

La **discretizzazione** è un'operazione di conversione di una variabile continua in una variabile discreta (es. categoria). Spesso la discretizzazione ha lo scopo di eseguire rappresentazioni grafiche come quella a istogramma. La funzione **cut**():

- suddivide il primo parametro (un array o una serie) in categorie semiaperte;
- il secondo parametro è un numero di categorie equamente spaziate o una lista di valori estremi delle categorie.

Se l'obiettivo è quello di suddividere la sequenza in **N categorie**, è possibile passare alla funzione cut() una lista di N+1 valori estremi. Le categorie prodotte da cut(), inoltre, appartengono a un tipo di dati ordinale. Esse sono, pertanto, classificabili e confrontabili tra loro. Il codice seguente mostra le prime righe

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

del risultato della discretizzazione in quattro categorie su una serie di elementi uniformemente distribuiti. Gli elementi di categorie saranno le rispettive categorie di s in base al valore assunto.

```
import pandas as pd
import numpy as np

n = np.random.uniform(0,100,20)
s = pd.Series(s)

categorie = pd.cut(s,4).head(7)
print(categorie)
```

```
0    (75.691, 99.986]
1    (51.396, 75.691]
2    (75.691, 99.986]
3    (27.101, 51.396]
4    (27.101, 51.396]
5    (51.396, 75.691]
6     (2.709, 27.101]
dtype: category
Categories (4, interval[float64]): [(2.709, 27.101] < (27.101, 51.396] < (51.396, 75.691] < (75.691, 99.986]]
```

Per le N categorie è possibile anche specificare le etichette attraverso il parametro opzionale **labels** come riportato nel codice seguente.

```
import pandas as pd
import numpy as np

n = np.random.uniform(0,100,20)
s = pd.Series(s)

categorie = pd.cut(s,3,labels=("Basso","Medio","Alto")).head(7)
print(categorie)
```

```
0     Alto
1     Alto
2     Alto
3    Medio
4    Medio
5    Medio
6    Basso
dtype: category
Categories (3, object): ['Basso' < 'Medio' < 'Alto']
```

Se, invece, si specifica *labels=False*, la funzione ***cut***() numera le categorie al posto di fornire loro un'etichetta come mostrato di seguito.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import pandas as pd
import numpy as np

n = np.random.uniform(0,100,20)
s = pd.Series(s)

categorie = pd.cut(s,3,labels=False).head(7)
print(categorie)
```

```
0    2
1    2
2    2
3    1
4    1
5    1
6    0
dtype: int64
```

La funzione **qcut**() è simile a ***cut***(), ma ragiona in termini di quantili (es. mediana, quartili), non di ampiezza delle categorie. Il codice seguente mostra l'uso di qcut() con tre quantili.

```
import pandas as pd
import numpy as np

n = np.random.uniform(0,100,20)
s = pd.Series(s)

quantili = pd.qcut(s,3,labels=('Basso','Medio','Alto')).head(7)
print(quantili)
```

```
0     Alto
1     Alto
2     Alto
3    Basso
4    Medio
5     Alto
6    Basso
dtype: category
Categories (3, object): ['Basso' < 'Medio' < 'Alto']
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Generazione di frame e operazioni con i file

3.1 Generazione di DataFrame

La generazione di DataFrame può avvenire in diversi modi. Si analizzano, in questo paragrafo, le diverse opzioni possibili che Pandas mette a disposizione.

Opzione 1: generazione di **DataFrame vuoti** come riportato nel codice seguente.

```
import pandas as pd

df = pd.DataFrame()
print(df)
```

```
Empty DataFrame
Columns: []
Index: []
```

Opzione 2: generazione di DataFrame costituiti da una sola colonna come riportato nel codice seguente.

```
import pandas as pd

df = pd.DataFrame(['Luca','Elena','Tommaso','Edoardo','Simona'])
print(df)
```

```
         0
0     Luca
1    Elena
2  Tommaso
3  Edoardo
4   Simona
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Opzione 3: generazione di **DataFrame per righe** assegnando nomi alle colonne attraverso l'argomento *columns* come riportato nel codice seguente.

```
import pandas as pd

df = pd.DataFrame(data =[['Luca',28,'Milano'],['Alessia',22,'Venezia'],\
                         ['Silvia',24,'Messina'],['Filippo',31,'Bari']],\
                  columns=['nome','eta','citta'])
print(df)
```

```
      nome  eta    citta
0     Luca   28   Milano
1  Alessia   22  Venezia
2   Silvia   24  Messina
3  Filippo   31     Bari
```

Opzione 4: generazione di **DataFrame da Series**. Nell'esempio seguente si creano due Series di nomi di capitali e popolazione di Stati. Il punto di contatto tra le due Series sono gli indici (uguali in entrambe e non necessariamente nello stesso ordine).

```
import pandas as pd

capitali = pd.Series(data=['Roma','Parigi', 'Madrid', 'Lisbona', 'Berna'],
                     index = ['Italia','Francia', 'Spagna', 'Portogallo', 'Svizzera'])

popolazione = pd.Series(data=[60665551,64291280,46926963,10608156,8077833],
                     index = ['Italia','Francia', 'Spagna', 'Portogallo', 'Svizzera'])

df = pd.DataFrame({'capitale':capitali, 'popolazione':popolazione})
print(df)
```

```
           capitale  popolazione
Italia         Roma     60665551
Francia      Parigi     64291280
Spagna       Madrid     46926963
Portogallo  Lisbona     10608156
Svizzera     Berna      8077833
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Opzione 5: generazione di **DataFrame da file CSV** (con la funzione **read_csv**()).

```
import pandas as pd
import numpy as np

df = pd.read_csv('persone.csv', delimiter = ';')
print(df)
```

```
      nome  cognome  altezza  peso     nascita sesso
0   Giulio    Rossi     1.84    89  1994-08-12     M
1  Alessia    Verdi     1.62    53  1993-03-07     F
2     Enea  Bianchi     1.78    73  1996-05-06     M
3    Paola     Neri     1.74    61  1999-09-30     F
4   Ilenia   gialli     1.70    70  1998-12-27     F
5  Valerio    Verdi     1.88    88  1996-05-06     M
```

Opzione 6: generazione di **DataFrame da file Excel** (con la funzione **read_excel**()).

```
import pandas as pd
import numpy as np

df = pd.read_excel('persone_excel.xlsx')
print(df)
```

```
      nome  cognome  altezza  peso    nascita sesso
0   Giulio    Rossi     1.84    89 1994-08-12     M
1  Alessia    Verdi     1.62    53 1993-03-07     F
2     Enea  Bianchi     1.78    73 1996-05-06     M
3    Paola     Neri     1.74    61 1999-09-30     F
4   Ilenia   gialli     1.70    70 1998-12-27     F
5  Valerio    Verdi     1.88    88 1996-05-06     M
```

Opzione 7: generazione di **DataFrame da database relazionale** Sqlite come query al database stesso. Per una maggiore chiarezza, questa opzione verrà illustrata contestualmente al salvataggio di un DataFrame su database relazionale.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3.2 Salvataggio di DataFrame su file

Anche per quanto riguarda il salvataggio di DataFrame, ci sono alcune opzioni diverse. Si analizzano, in questo paragrafo, le diverse opzioni possibili che Pandas mette a disposizione.

Opzione 1: salvataggio di DataFrame su file CSV attraverso la funzione **to_csv**(). Prima di procedere al salvataggio del file CSV, bisogna effettuare alcune scelte:

- se si vuole scegliere un separatore, esso va indicato come argomento *sep* (di default il separatore è la virgola);
- per riportare intestazioni del DataFrame su file, argomento *header = True*, altrimenti *header = False*;
- per non salvare gli indici del DataFrame su file, argomento *index=None*.

Nel caso riportato, si genera il file file_backup.csv con il punto e virgola come separatore, rinunciando ad indici e intestazioni:

```
df.to_csv('file_backup.csv', index = None, sep=';' , header=False)
```

Opzione 2: salvataggio di **DataFrame su file Excel**. I passi da eseguire per poter salvare un DataFrame in un file Excel sono i seguenti:

- si crea un'istanza dell'oggetto di tipo *ExcelWriter* che usa come engine la libreria *xlsxwriter;*
- tramite gli argomenti della funzione **to_excel**() si specifica di non volere salvare gli indici (*index = False*) e che il foglio della cartella Excel creata deve chiamarsi *Persone;*
- infine, con il metodo **save**() dell'oggetto writer si crea il file *dati_excel.xlsx*.

```
writer = pd.ExcelWriter('dati_excel.xlsx', engine='xlsxwriter')
df.to_excel(writer, index=False, sheet_name='Persone')
writer.save()
```

Opzione 3: salvataggio di **DataFrame su database relazionale.** Per ottenere ciò, si utilizza l'*engine* messo a disposizione da *SqlAlchemy* (www.sqlalchemy.org). Come primo passo, si prepara una connessione ad un database Sqlite (database relazionale SQL autonomo che non necessita di server e configurazione) come indicato nel seguente codice:

```
from sqlalchemy import create_engine
engine = create_engine('sqlite:///database_nuovo.db')
```

Successivamente, sarà sufficiente invocare la funzione **to_sql**() per salvare il DataFrame *df* nella tabella persone senza memorizzazione degli indici.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
df.to_sql('persone',engine,index = False)
```

Sfruttando la stessa connessione dell'oggetto *engine* e la funzione di Pandas ***read_sql***(), è possibile svolgere operazioni di lettura in linguaggio SQL al fine di **interrogare il database relazionale per ottenere un DataFrame**. Nel caso seguente, si vuole memorizzare nel DataFrame l'elenco dei cognomi con il conteggio della loro presenza tra i dati.

```
query_SQL = """SELECT cognome, COUNT(*) AS quanti
FROM persone
GROUP BY cognome
ORDER BY quanti DESC"""
df_new= pd.read_sql(query_SQL,engine)
print(type(df_new))
print(df_new)
```

```
<class 'pandas.core.frame.DataFrame'>
   cognome  quanti
0    Verdi       2
1   gialli       1
2    Rossi       1
3     Neri       1
4  Bianchi       1
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Grafici con Pandas

Con Pandas, l'impiego di grafici è più immediato che con NumPy. Pandas, infatti, contiene già tutto il codice necessario attivato mediante Matplotlib.

Si caricano, ora, da file csv i dati con Pandas di dati finanziari generati casualmente, ma distribuiti in quattro colonne. Ogni riga descrive un'ipotetica giornata di borsa e le colonne sono rispettivamente:

- valore di apertura di un titolo azionario;
- valore di chiusura;
- valore massimo di giornata;
- minimo toccato nella stessa giornata.
-

```
import pandas as pd

df = pd.read_csv('dati_finanziari_grafico.csv', delimiter = ';')
print(df)
```

```
       apertura     chiusura      massimo       minimo
0   2877.081209  2915.098802  3024.925509  2934.292148
1   2961.662414  2802.526556  3170.191768  3027.946838
2   2976.009877  3053.688219  3082.056871  3010.465456
3   3001.122900  2985.613949  3115.679940  3017.236826
4   3064.763912  2750.014024  2861.106227  2661.106227
..          ...          ...          ...          ...
95  2941.266144  3165.405391  3013.346997  2813.346997
96  3056.533495  3074.327383  3072.299998  3004.494837
97  2958.712671  2863.109289  3104.803863  3054.555508
98  2923.899793  3114.659623  3010.335842  2810.335842
99  3070.596217  3027.349904  3047.022373  2917.049375

[100 rows x 4 columns]
```

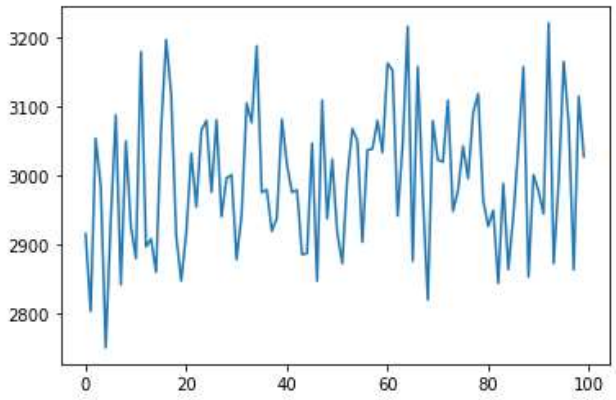
I DataFrame Pandas contengono già i metodi per l'elaborazione dei grafici. È, infatti, possibile visualizzare il grafico delle chiusure di df con il metodo **plot**() come mostrato di seguito.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import pandas as pd
import matplotlib.pyplot as matplot

df = pd.read_csv('dati_finanziari_grafico.csv', delimiter = ';')
df['chiusura'].plot(kind='line')
matplot.show()
```



Con l'argomento **kind** si specifica che tipo di grafico si desidera visualizzare. Nel caso mostrato, infatti, *line* sta per grafico lineare.

Per ampliare la trattazione dei grafici con Pandas, si calcola, ora, una media mobile a 10 giorni rappresentandola come linea secondaria a quella già visualizzata in precedenza.

Considerando che la serie delle chiusure risiede in *df['chiusura']*, la sua media mobile a dieci giorni può essere ottenuta nel seguente modo:

```
r = df['chiusura'].rolling(window=10).mean()
print(r[:16])
```

```
0             NaN
1             NaN
2             NaN
3             NaN
4             NaN
5             NaN
6             NaN
7             NaN
8             NaN
9     2934.774070
10    2931.218516
11    2968.913316
12    2953.208121
13    2945.425838
14    2956.391728
15    2969.494799
Name: chiusura, dtype: float64
```

Essendo la media mobile ottenuta una Series, se ne può richiamare il grafico con **plot**() ottenendo i due grafici contemporaneamente.

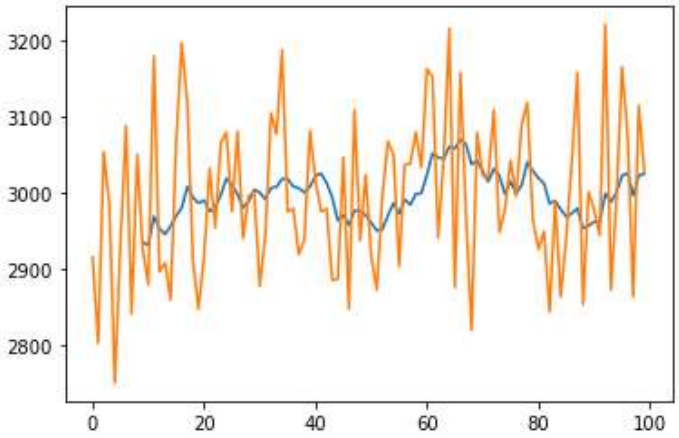
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import pandas as pd
import matplotlib.pyplot as matplot

df = pd.read_csv('dati_finanziari_grafico.csv', delimiter = ';')

r = df['chiusura'].rolling(window=10).mean()
r.plot(kind='line')
df['chiusura'].plot(kind='line')
matplot.show()
```



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Esercizio di Data Science

S propone un esercizio di analisi di transazioni bancarie[1]. Si hanno a disposizione due file CSV riguardanti transazioni bancarie. In particolare:

- il file ***conti.csv*** che contiene un elenco di conti correnti per ognuno dei quali vengono riportati **numero e codice della filiale bancaria**;
- il file ***transazioni.csv*** che riporta un elenco di **transazioni** operate sui vari conti complete di **data, importo, causale e identificativo del conto**.

Per i codici causale si utilizzano quattro stringhe:

1. PAGO per Pagobancomat;
2. BODA per bonifico in entrata;
3. BNFC per bonifico in uscita;
4. VERS per versamento in contanti.

Per maggiore chiarezza, si riportano, di seguito, i due file CSV.

conti.csv

```
conto;filiale
400500;IT001
400400;IT001
400300;IT002
400200;IT003
400100;IT002
```

transazioni.csv

```
data;importo;causale;conto
2019-01-21;750;VERS;400400
2019-01-26;-300;BNFC;400100
2019-02-02;450;BODA;400400
2019-02-05;670;BODA;400200
2019-02-18;100;VERS;400400
2019-03-18;-650;BNFC;400200
2019-03-27;-25;PAGO;400400
2019-03-29;1100;VERS;400400
2019-04-03;-210;BNFC;400400
2019-04-03;-520;BNFC;400200
2019-04-12;840;VERS;400500
2019-05-18;1000;BNFC;400200
2019-05-18;300;VERS;400100
2019-05-31;510;VERS;400100
2019-06-06;800;VERS;400500
2019-06-16;600;BODA;400500
2019-07-09;-88;PAGO;400500
2019-07-09;-300;BNFC;400100
2019-08-14;-120;PAGO;400500
2019-08-21;-1100;BNFC;400200
```

[1] Esercizio tratto dal libro Maggi, G. (2020). Data science con Python. Edizioni LSWR.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Obiettivi dell'esercitazione sono:

1. calcolare un saldo per conto corrente;
2. calcolare un saldo per tipologia di operazione, ma suddivisa per conti;
3. recuperare le operazioni appartenenti ad una specifica filiale;
4. eseguire un saldo per tipologia di operazione, suddivisa per filiale;
5. individuare eventuali conti di cui non risultano operazioni.

Per raggiungere tutti gli obiettivi, sarà necessario sfruttare tutte le conoscenze di Pandas tra cui selezioni, raggruppamenti e JOIN.

Per prima cosa si caricano i conti e le transazioni in due DataFrame come mostrato di seguito.

```
import pandas as pd

def caricamento_dati():
    conti = pd.read_csv('conti.csv',sep=';')
    transazioni = pd.read_csv('transazioni.csv',sep=';')
    return conti, transazioni

def main():
    conti, transazioni = caricamento_dati()
    print("Conti")
    print(conti)
    print()
    print("Transazioni")
    print(transazioni)
main()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Conti

	conto	filiale
0	400500	IT001
1	400400	IT001
2	400300	IT002
3	400200	IT003
4	400100	IT002

Transazioni

	data	importo	causale	conto
0	2019-01-21	750	VERS	400400
1	2019-01-26	-300	BNFC	400100
2	2019-02-02	450	BODA	400400
3	2019-02-05	670	BODA	400200
4	2019-02-18	100	VERS	400400
5	2019-03-18	-650	BNFC	400200
6	2019-03-27	-25	PAGO	400400
7	2019-03-29	1100	VERS	400400
8	2019-04-03	-210	BNFC	400400
9	2019-04-03	-520	BNFC	400200
10	2019-04-12	840	VERS	400500
11	2019-05-18	1000	BNFC	400200
12	2019-05-18	300	VERS	400100
13	2019-05-31	510	VERS	400100
14	2019-06-06	800	VERS	400500
15	2019-06-16	600	BODA	400500
16	2019-07-09	-88	PAGO	400500
17	2019-07-09	-300	BNFC	400100
18	2019-08-14	-120	PAGO	400500
19	2019-08-21	-1100	BNFC	400200

Procediamo, progettando e scrivendo il codice necessario per raggiungere i 5 obiettivi prefissati, obiettivo per obiettivo.

Obiettivo 1: *calcolare il saldo per il conto corrente.*

Per calcolare la somma delle transazioni suddividendole per conto corrente è sufficiente il DataFrame *transazioni* che contiene i campi **data, importo, causale, conto**. Si dovrà, quindi, eseguire un **raggruppamento per numero di *conto*** e richiedere la **somma del campo *importo*** come indicato nel codice seguente (funzione saldo_per_conto()).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def saldo_per_conto(transazioni):
    return transazioni.groupby('conto')['importo'].sum()

def main():
    conti, transazioni = caricamento_dati()
    print(saldo_per_conto(transazioni))

main()
```

```
conto
400100     210
400200    -600
400400    2165
400500    2032
Name: importo, dtype: int64
```

Obiettivo 2: *calcolare il saldo per tipologia di operazione suddivisa per conti.*

Il saldo calcolato al punto precedente può essere ulteriormente dettagliato eseguendo un **raggruppamento doppio sia per conto sia per causale**. Si dovrà, quindi, passare al groupby() una lista di due campi ['conto','causale'] nella funzione saldo_per_conto_causale() riportata di seguito. In questo modo, si avranno indicazioni sulle operazioni svolte sui singoli conto corrente e il loro importo per categoria.

```
def saldo_per_conto_causale(transazioni):
    return transazioni.groupby(['conto','causale'])['importo'].sum()

def main():
    conti, transazioni = caricamento_dati()
    print(saldo_per_conto_causale(transazioni))

main()
```

```
conto   causale
400100  BNFC       -600
        VERS        810
400200  BNFC      -1270
        BODA        670
400400  BNFC       -210
        BODA        450
        PAGO        -25
        VERS       1950
400500  BODA        600
        PAGO       -208
        VERS       1640
Name: importo, dtype: int64
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Rileggendo l'output riga per riga avremo indicazioni su quali sono state le operazioni svolte sui singoli conti correnti e il loro importo per categoria. Per esempio le prime righe sono:

```
400100          BNFC    -600
                VERS     810
```

e indicano che sul conto 400100 sono stati svolti bonifici in uscita (codice BNFC) e versamenti in contanti (codice VERS). I primi ammontano a 600,00 euro (segno negativo in quanto esborsi), i secondi a 810,00 euro (segno positivo in quanto accrediti). Deve comunque emergere un'assoluta coerenza con quanto rilevato con l'obiettivo precedente in quanto si tratta delle medesime operazioni, ma raggruppate in modo differente. Il saldo complessivo del conto 400100, infatti, risultava essere in attivo di 210,00 euro che equivale, appunto, alla differenza tra le due voci qui riscontrate: accredito di 810,00 per versamenti contanti e uscite per 600,00 per bonifici emessi.

Questo esercizio dimostra come il raggruppamento sia fondamentale per ottenere lavorazioni sui dati e con una giusta scelta dei campi da coinvolgere nel *groupby*() è possibile ottenere la granularità desiderata.

Obiettivo 3: *recuperare le operazioni di una specifica filiale.*

Per raggiungere questo obiettivo, risulta chiara l'esigenza di dover ricorrere ad **un'operazione di JOIN** in quanto:

- le informazioni riferite alle filiali sono nel DataFrame *conti*;
- le informazioni riferite alle operazioni sono nel DataFrame *transazioni.*

Si procede, quindi, realizzando il JOIN che permette di ottenere il «super» DataFrame con tutte le informazioni necessarie come indicato di seguito.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
conti_transazioni = pd.merge(transazioni, conti, on='conto')
print(conti_transazioni)
```

```
         data  importo causale   conto filiale
0  2019-01-21      750    VERS  400400   IT001
1  2019-02-02      450    BODA  400400   IT001
2  2019-02-18      100    VERS  400400   IT001
3  2019-03-27      -25    PAGO  400400   IT001
4  2019-03-29     1100    VERS  400400   IT001
5  2019-04-03     -210    BNFC  400400   IT001
6  2019-01-26     -300    BNFC  400100   IT002
7  2019-05-18      300    VERS  400100   IT002
8  2019-05-31      510    VERS  400100   IT002
9  2019-07-09     -300    BNFC  400100   IT002
10 2019-02-05      670    BODA  400200   IT003
11 2019-03-18     -650    BNFC  400200   IT003
12 2019-04-03     -520    BNFC  400200   IT003
13 2019-05-18     1000    BNFC  400200   IT003
14 2019-08-21    -1100    BNFC  400200   IT003
15 2019-04-12      840    VERS  400500   IT001
16 2019-06-06      800    VERS  400500   IT001
17 2019-06-16      600    BODA  400500   IT001
18 2019-07-09      -88    PAGO  400500   IT001
19 2019-08-14     -120    PAGO  400500   IT001
```

Partendo dal DataFrame ottenuto attraverso la JOIN, si procede filtrando le operazioni relative ad una filiale specifica (es. 'IT001') con la funzione *transazioni_filiale*() riportata di seguito.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def transazioni_filiale(transazioni,conti,filiale):
    conti_transazioni = pd.merge(transazioni, conti, on='conto')
    tf = conti_transazioni[conti_transazioni['filiale']==filiale]
    return tf

def main():
    conti, transazioni = caricamento_dati()
    tf = transazioni_filiale(transazioni,conti,'IT001')
    print(tf)

main()
```

```
          data  importo causale   conto filiale
0   2019-01-21      750    VERS  400400   IT001
1   2019-02-02      450    BODA  400400   IT001
2   2019-02-18      100    VERS  400400   IT001
3   2019-03-27      -25    PAGO  400400   IT001
4   2019-03-29     1100    VERS  400400   IT001
5   2019-04-03     -210    BNFC  400400   IT001
15  2019-04-12      840    VERS  400500   IT001
16  2019-06-06      800    VERS  400500   IT001
17  2019-06-16      600    BODA  400500   IT001
18  2019-07-09      -88    PAGO  400500   IT001
19  2019-08-14     -120    PAGO  400500   IT001
```

Di tutti i conti esistenti solo il 400400 e il 400500 appartengono alla filiale IT001 ed, infatti, le transazioni che appaiono nel risultato si riferiscono a tali conti.

Obiettivo 4: *eseguire il saldo per tipologia di operazione per ogni filiale.*

Questo esercizio lega tra loro le modalità precedenti. In particolare, a seguito del JOIN tra conti e transizioni, è necessario eseguire un raggruppamento su due campi (filiale, causale) e sommare gli importi come riportato di seguito nella funzione *saldo_filiale_causale*().

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def saldo_filiale_causale(transazioni, conti):
    conti_transazioni = pd.merge(transazioni, conti, on='conto')
    saldi = conti_transazioni.groupby(['filiale','causale'])['importo'].sum()
    return saldi

def main():
    conti, transazioni = caricamento_dati()
    print(saldo_filiale_causale(transazioni,conti))

main()
```

```
filiale  causale
IT001    BNFC       -210
         BODA       1050
         PAGO       -233
         VERS       3590
IT002    BNFC       -600
         VERS        810
IT003    BNFC      -1270
         BODA        670
Name: importo, dtype: int64
```

Obiettivo 5: *individuare eventuali conti in cui non sono state svolte operazioni*.

In questo caso non sarà necessario eseguire né JOIN né raggruppamenti. Per raggiungere l'obiettivo, infatti, si farà uso dell'**operatore isin**() che permette, in una selezione, di verificare quale elemento di una serie fa parte di un'altra serie.

Dunque, si chiede di verificare quali conto corrente del DataFrame *conti* non sono presenti nel DataFrame *transazioni*. Si precisa che per applicare la **negazione** al fine di invertire lo stato logico dell'operatore *isin*() è necessario utilizzare l'**operatore tilde ~**(). Procediamo con la scrittura della funzione *conti_inattivi*() come mostrato di seguito. Il risultato rivelerà che il conto numero 400300 risulta inattivo.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def conti_inattivi(transazioni,conti):
    #array booleano dei conti presenti in transazioni
    ca = conti['conto'].isin(transazioni['conto'])
    #estrazione dei conti non presenti in transazioni (inattivi)
    ci = conti['conto'][~(ca)]
    return ci

def main():
    conti, transazioni = caricamento_dati()
    print("Conti inattivi")
    print(conti_inattivi(transazioni,conti))

main()
```

```
Conti inattivi
2    400300
Name: conto, dtype: int64
```

A scopo riepilogativo, si riporta il codice che permette di raggiungere tutti i 5 obiettivi dell'esercizio di analisi di transazioni bancarie. In particolare, il codice riporta le funzioni analizzate concatenate in una serie di invocazioni in modo tale che vengano attivate in un'unica esecuzione.

```
import pandas as pd

def caricamento_dati():
    conti = pd.read_csv('conti.csv',sep=';')
    transazioni = pd.read_csv('transazioni.csv',sep=';')
    return conti, transazioni

def saldo_per_conto(transazioni):
    return transazioni.groupby('conto')['importo'].sum()

def saldo_per_conto_causale(transazioni):
    return transazioni.groupby(['conto','causale'])['importo'].sum()

def transazioni_filiale(transazioni,conti,filiale):
    conti_transazioni = pd.merge(transazioni, conti, on='conto')
    tf = conti_transazioni[conti_transazioni['filiale']==filiale]
    return tf

def saldo_filiale_causale(transazioni, conti):
    conti_transazioni = pd.merge(transazioni, conti, on='conto')
    saldi = conti_transazioni.groupby(['filiale','causale'])['importo'].sum()
    return saldi

def conti_inattivi(transazioni,conti):
    #array booleano dei conti presenti in transazioni
    ca = conti['conto'].isin(transazioni['conto'])
    #estrazione dei conti non presenti in transazioni (inattivi)
    ci = conti['conto'][~(ca)]
    return ci
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def main():
    conti, transazioni = caricamento_dati()
    print("1. Saldo dei conto corrente")
    print(saldo_per_conto(transazioni))
    print()
    print("""2. Saldo per tipologia di operazione
          suddivisa per conto corrente""")
    print(saldo_per_conto_causale(transazioni))
    print()
    tf = transazioni_filiale(transazioni,conti,'IT001')
    print("3. Operazioni della filiale IT001")
    print(tf)
    print()
    print("4. Saldo per tipologia di operazione per ciascuna filiale")
    print(saldo_filiale_causale(transazioni,conti))
    print()
    print("5. Conti inattivi")
    print(conti_inattivi(transazioni,conti))

main()
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Zinoviev D. (2017). Data science con Python. Apogeo.
- Maggi G. (2020). Data science con Python. Edizioni LSWR.
- Marin I., Shukla A., Sarang VK. (2019). L'analisi dei Big Data con Python. Tecniche nuove.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).